

Quantification d'incertitude et éléments finis III

Philippe-André Luneau,
stagiaire El Mehdi En-Nahas

Été 2026

Résumé

Ce stage a pour but d'explorer comment des moyens de quantification d'incertitude, d'analyse de sensibilité et d'analyse de fiabilité utilisant des approches de type Monte Carlo/statistique/probabiliste peuvent être intégrés à l'environnement MEF++. Notamment, l'avènement de mefpp4py, l'interface Python de MEF++, permettra de faciliter fortement les traitements de type statistique.

L'essentiel de la méthode SSFEM a été codée dans mefpp4py par Augustin Perrin (stage 2025). Le problème pour l'instant est principalement dans le calcul numérique de la décomposition Karhunen-Loève, qui n'est pas adapté à un logiciel d'éléments finis classique comme MEF++ (équation non-locale). On doit principalement trouver comment contourner ce problème afin de pouvoir générer des cas tests en 2D et 3D (on a présentement seulement des cas 1D où des solutions analytiques existent pour l'équation de Fredholm avec noyau Gaussien).

Lien pour accéder au Overleaf de ce document : [Overleaf](#).

Table des matières

1	Équation de diffusion avec coefficient stochastique	3
2	Projection de Galerkin, éléments finis	3
2.0.1	Existence et unicité de la solution au problème faible, orthogonalité de Galerkin	4
3	Méthode SSFEM	5
3.0.1	Obtention de D_i et λ_i	6
4	Livrables	7
5	Avancées	7
	Annexes	9
A	blabla	9

1 Équation de diffusion avec coefficient stochastique

On se donne un domaine Ω ouvert, borné et connexe de $\mathbb{R}^2$. On considère le problème de Dirichlet suivant :

$$-\nabla \cdot (D(x, \theta)u(x)) = f(x) \quad \text{sur } \Omega \quad (1)$$

$$u(x) = g(x) \quad \text{sur } \partial\Omega \quad (2)$$

où $f \in L^2(\Omega)$ et $g(x) \in L^2(\partial\Omega)$. On cherchera une solution $u \in H^1(\Omega)$.

Soit un espace probabilisé $(\Theta, \mathcal{S}, \mu)$ on définit la famille de variable aléatoire $D(x, \cdot) \in L^2_\mu(\Theta)$ (en fait, c'est un processus aléatoire sur Ω). On note que $L^2_\mu(\Theta)$ est un espace de Hilbert avec le produit scalaire $\langle f, g \rangle = \mathbb{E}[fg] = \int_{\mathbb{R}} fg d\mu$.

En terme de modélisation, le coefficient D représente le coefficient de diffusion ; il indique l'intensité de l'uniformisation locale typique d'un processus de diffusion. Le fait qu'il soit une variable aléatoire peut représenter, par exemple, une incertitude dans les propriétés du matériau (on peut supposer que le matériau a une propriété macro, mais au niveau micro, il peut y avoir des imperfections). La solution de l'équation devient dans ce cas elle-même une variable aléatoire. On s'intéressera généralement à estimer des statistiques par rapport à cette solution, notamment les moments (e.g. l'espérance $\mathbb{E}[u(x)]$ ou la variance $\text{Var}[u(x)]$) dans le but de prédire en moyenne comment la solution se comporte. L'estimation de la variance est lié au domaine de la **quantification d'incertitude**, très important d'un point de vue industriel.

2 Projection de Galerkin, éléments finis

Mettons de côté la stochasticité de D , i.e. $D = D(x)$, et supposons $g = 0$ (il est possible de travailler dans un espace quotienté pour prendre un g non-nul, mais cela simplifie cette brève présentation). Pour résoudre une équation de ce type numériquement, la méthode des éléments finis est une des plus utilisées. Sachant que la solution u est dans un espace de Hilbert V (ici $V = H_0^1(\Omega)$), on construit un sous-espace V_h de dimension n à l'aide d'une base donnée $\Phi = \{\varphi_i : i = 1, \dots, n\}$. On cherchera la solution approximative dans ce sous-espace (c'est ce qu'on appelle une projection de Galerkin).

Premièrement, on doit considérer que les dérivées sont prises **au sens faible** (au sens des distributions). Cela nous permet, en passant par les formules de Green (intégration par partie pour le calcul vectoriel), d'obtenir la **formulation faible** du problème de diffusion : trouver $u \in V$ t.q.

$$\int_{\Omega} D \nabla u \cdot \nabla v dx = \int_{\Omega} f v dx, \quad \forall v \in V. \quad (3)$$

L'approche Galerkin dit de restreindre ce problème à l'espace V_h : trouver $u_h \in V_h$ t.q.

$$a(u_h, v_h) := \int_{\Omega} D \nabla u_h \cdot \nabla v_h dx = \int_{\Omega} f v_h dx =: \ell(v_h), \quad \forall v_h \in V_h. \quad (4)$$

Maintenant, on sait que l'inconnue admet une représentation comme combinaison linéaire des fonc-

tions de base :

$$u_h(x) = \sum_{i=1}^n U_i \varphi_i(x)$$

où $U = (U_1, \dots, U_n)$ sont les coordonnées de u_h dans la base de V_h . On remarque que a est bilinéaire et ℓ est linéaire. On peut donc réécrire le problème comme

$$\sum_{i=1}^n a(\varphi_i, v_h) U_i = \ell(v_h), \quad \forall v_h \in V.$$

En particulier, on peut créer n équations en choisissant successivement $v_h = \varphi_j$:

$$\sum_{i=1}^n a(\varphi_i, \varphi_j) U_i = \ell(\varphi_j), \quad j = 1, \dots, n.$$

Ceci est un système linéaire à n équations et n inconnues, on peut donc l'écrire sous forme matricielle, $AU = F$ où $a_{ij} = a(\varphi_i, \varphi_j)$ et $f_i = \ell(\varphi_i)$. Maintenant que le système est assemblé, il suffit de le résoudre pour obtenir l'approximation.

La **méthode des éléments finis** donne un algorithme systématique et efficace pour construire la base Φ à partir d'un découpage du domaine Ω en plusieurs petits sous-domaines appelés **éléments**, et de polynômes d'interpolations, typiquement les polynômes de Lagrange. Sa grande généralité lui permet d'attaquer des problèmes variationnels elliptiques sur des géométries presque arbitraires, contrairement à des méthodes comme les différences finies dont l'implémentation sur des géométries non-rectangulaires est extrêmement fastidieuse.

2.0.1 Existence et unicité de la solution au problème faible, orthogonalité de Galerkin

L'existence et unicité de la solution au problème faible découlent directement des propriétés de a et ℓ (continuité et coercivité) et du fait que V est un espace de Hilbert. Le théorème de Lax-Milgram garantit l'existence et l'unicité sous ces conditions.

De plus, sous ces conditions, $a(\cdot, \cdot)$ est en fait un produit scalaire, et on comprend en fait que la solution au problème est en fait la projection orthogonale de u sur V_h dans le produit scalaire a :

$$a(u_h, v) = \ell(v) = a(u, v) \quad \forall v \in V \iff a(u_h - u, v) = 0 \quad \forall v \in V.$$

Cette propriété est l'orthogonalité de Galerkin, i.e. l'erreur d'approximation est orthogonale à V_h , et donc l'approximation est optimale. Le problème est en fait un problème d'optimisation : on peut en effet montrer qu'il minimise une fonctionnelle, l'énergie de Dirichlet,

$$J(u) = \frac{1}{2} a(u, u) - \ell(u).$$

Montrer que cette fonctionnelle a un unique minimum est une autre façon de montrer l'existence et l'unicité (et l'optimisation est aussi une autre façon d'obtenir numériquement une approximation de la solution).

3 Méthode SSFEM

La méthode SSFEM (*spectral stochastic finite element method*) est une extension de la méthode des éléments finis pour la résolution d'EDPs avec des coefficients aléatoires comme présentées ci-haut.

Le coefficient aléatoire est exprimé dans une base de $L^2_\mu(\Theta)$ par le développement de Karhunen-Loève :

$$D(x, \theta) = D_0(x) + \sum_{i=1}^{\infty} \sqrt{\lambda_i} D_i(x) \xi_i(\theta) \approx D_0(x) + \sum_{i=1}^N \sqrt{\lambda_i} D_i(x) \xi_i(\theta)$$

où ξ_i sont de moyenne 0, deux à deux indépendantes et de variance 1. Le terme D_0 représente ainsi la moyenne de D (car $\mathbb{E}[\xi_i] = 0$).

La forme bilinéaire a du problème faible de la section précédente devient

$$\int_{\Omega} D_0 \nabla \varphi_i \cdot \nabla \varphi_j dx + \sum_{m=1}^N \xi_m(\theta) \int_{\Omega} D_m(x) \nabla \varphi_i \cdot \nabla \varphi_j dx$$

ou de manière plus compacte, la matrice globale du problème stochastique est

$$A(\theta) = A_0 + \sum_{m=1}^N \xi_m(\theta) A_m$$

où A_m est la matrice globale du problème pour lequel on utilise D_m comme coefficient de diffusion.

Par la suite, on utilise une approximation en chaos polynomial pour représenter $u_h(x, \theta) = \sum_i U_i(\theta) \phi_i(x)$, spécifiquement le coefficient aléatoire (celui qui dépend de θ). En effet, on peut choisir une base de polynômes $L^2_\mu(\theta)$ -orthogonaux $\Psi = \{\psi_i : i \in \mathbb{N}\}$ et écrire

$$U(\theta) \approx \sum_{i=0}^{\infty} \widehat{U}_i \underbrace{\psi_i(\xi_1(\theta), \dots, \xi_N(\theta))}_{\psi_i(\theta)} \approx \sum_{i=0}^P \widehat{U}_i \psi_i(\theta)$$

où $\widehat{U}_j \in \mathbb{R}^n$ sont inconnus. La base Ψ doit aussi respecter la propriété d'être deux à deux indépendants, et d'être de moyenne nulle (sauf $\psi_0 \equiv 1$). Cependant, la représentation sera inexacte car on tronque la série. On va donc choisir les vecteurs $\widehat{U}_i$ en vérifiant que le résidu (erreur à posteriori) sera orthogonal à l'espace engendré par les $P+1$ polynômes (même idée que la projection de Galerkin). On veut donc imposer l'orthogonalité au sens du produit scalaire de $L^2_\mu(\Theta)$, i.e.

$$\underbrace{\mathbb{E}[(F - A(\theta)U(\theta)) \psi_k]}_{\text{résidu}} = 0, \quad k = 0, \dots, P.$$

En développant, on obtient un système linéaire à résoudre (on utilise la convention $\xi_0 \equiv 1$) : pour chaque $k = 0, \dots, P$

$$\mathbb{E} \left[F \psi_k(\theta) - \sum_{i=0}^N \sum_{j=0}^P \xi_i(\theta) \psi_j(\theta) \psi_k(\theta) A_i \widehat{U}_j \right] = \mathbb{E}[\psi_k] F - \sum_{i=0}^N \sum_{j=0}^P \mathbb{E}[\xi_i \psi_j \psi_k] A_i \widehat{U}_j$$

On note $\mathcal{F}_k = \mathbb{E}[\psi_k]F$ et $\mathcal{A}_{jk} = \sum_{i=0}^N \mathbb{E}[\xi_i \psi_j \psi_k] A_i$, ce qui donne une matrice $(P+1)n \times (P+1)n$ (n est la dimension de l'espace V_h)

$$\mathcal{A} = \begin{pmatrix} \mathcal{A}_{00} & \dots & \mathcal{A}_{0P} \\ \vdots & \ddots & \vdots \\ \mathcal{A}_{P0} & \dots & \mathcal{A}_{PP} \end{pmatrix}$$

et un vecteur $\mathcal{F} = (\mathcal{F}_1, \dots, \mathcal{F}_P)^\top$, et la solution sera $\mathcal{U} = (\hat{U}_1, \dots, \hat{U}_P)^\top$, i.e.

$$\mathcal{A}\mathcal{U} = \mathcal{F} \quad (5)$$

La solution approximative $u_h(x, \theta)$ est donnée par

$$u_h(x, \theta) = \sum_{i=1}^n \sum_{j=0}^P (\hat{U}_j)_i \psi_j(\theta) \varphi_i(x)$$

Par la suite, on obtient trivialement l'espérance

$$\mathbb{E}_\theta[u_h(x, \theta)] = \sum_{i=1}^n \sum_{j=0}^P (\hat{U}_j)_i \mathbb{E}[\psi_j] \varphi_i(x) = \sum_{i=1}^n (\hat{U}_0)_i \varphi_i(x)$$

et on pourrait faire de même pour la covariance.

3.0.1 Obtention de D_i et λ_i

Cela dépend du choix de processus stochastique pour $D(x, \theta)$. Si le processus est Gaussien, on sait que $\xi_i \sim \mathcal{N}(0, 1)$. Les D_i et λ_i seront les fonctions et valeurs propres respectivement du noyau d'auto-corrélation $K(x, y)$, i.e. la fonction qui détermine la corrélation entre $D(x, \theta)$ et $D(y, \theta)$ pour chaque paire $x, y \in \Omega$. Ce problème peut être résolu numériquement par la projection de Galerkin d'une équation de Fredholm sur V_h , donnant lieu à un problème de valeurs propres généralisé discret :

$$\int_{\Omega} K(x, y) D_i(x) dx = \lambda_i D_i(y) \approx \mathbf{K} \vec{d} = \lambda \mathbf{B} \vec{d}$$

Un premier challenge sera la construction "simple" de la matrice $\mathbf{K}$. Comment le faire potentiellement avec seulement les matrices A_i ? Peut-on utiliser des stratégies comme une approximation avec les *random Fourier features* ?

Stratégie à essayer :

1. Écrire proprement la projection de Galerkin de cette équation de Fredholm (intégrale double).
2. Substituer le noyau par une approximation *random Fourier features*.
3. Essayer de tout écrire en termes de A_i .
4. Identifier les composantes/blocs de $\mathbf{K}, \mathbf{B}$
5. Utiliser `mefpp4py` pour assembler ces matrices
6. Utiliser un solveur `SLEPc` (dans le `.champs` ?) pour résoudre et trouver d, λ
7. Valider la convergence en fonction du nombre de fréquences pour le cas 1D
8. valider en faisant des expériences stochastiques Monte Carlo à partir de valeurs générées par convolution

4 Livrables

Un code python utilisant mefpp4py LISIBLE et GÉNÉRIQUE qui permettra de générer un test multidimensionnel de la méthode SSFEM.

5 Avancées

Le chemin propre :

Si $K(x, y) = r(x - y)$ est une fonction radiale positive (noyau stationnaire) telle que $r(0) = 1$, on peut écrire le théorème de Bochner

$$K(x, y) = \int e^{-2\pi i \omega^\top (x-y)} p(\omega) d\omega \underset{\text{parité de } p}{=} \int \cos(2\pi \omega^\top (x - y)) p(\omega) d\omega.$$

On approxime l'intégrale par une quadrature, soit Monte Carlo ou quadrature déterministe. Dans tous les cas,

$$K(x, y) \approx \sum_{i=1}^N \beta_i \cos(2\pi \omega_i^\top (x - y)) =: K_N(x, y).$$

On pose une décomposition de type Fourier pour le champ aléatoire (aussi motivée par le théorème de Bochner) :

$$\delta D(x, \omega) \approx \sum_{i=1}^N a_i(\omega) \cos(2\pi \omega_i^\top x) + b_i(\omega) \sin(2\pi \omega_i^\top x).$$

On veut que le noyau de covariance du processus correspondent à K :

$$\begin{aligned} K(x, y) &= \text{Cov}[x, y] = \mathbb{E}_\omega[\delta D(x, \omega) \delta D(y, \omega)] \\ &\approx \sum_{i,j=1}^N \left(\cos(2\pi \omega_i^\top x) \cos(2\pi \omega_j^\top y) \mathbb{E}[a_i a_j] + \cos(2\pi \omega_i^\top x) \sin(2\pi \omega_j^\top y) \mathbb{E}[a_i b_j] \right. \\ &\quad \left. + \sin(2\pi \omega_i^\top x) \cos(2\pi \omega_j^\top y) \mathbb{E}[a_j b_i] + \sin(2\pi \omega_i^\top x) \sin(2\pi \omega_j^\top y) \mathbb{E}[b_i b_j] \right). \end{aligned}$$

Or, on fait les hypothèses :

- $\mathbb{E}[a_i] = \mathbb{E}[b_i] = 0$,
- $a_i \perp a_j, \quad b_i \perp b_j \quad (i \neq j)$,
- $a_i \perp b_j$,
- $\mathbb{E}[a_i^2] = \mathbb{E}[b_i^2] = \gamma_i$.

Par la formule d'addition des angles pour le cosinus,

$$\begin{aligned}
K_N(x, y) &= \sum_{i=1}^N \left(\cos(2\pi\omega_i^\top x) \cos(2\pi\omega_i^\top y) \mathbb{E}[a_i^2] + \sin(2\pi\omega_i^\top x) \sin(2\pi\omega_i^\top y) \mathbb{E}[b_i^2] \right) \\
&= \sum_{i=1}^N \gamma_i \left(\cos(2\pi\omega_i^\top x) \cos(2\pi\omega_i^\top y) + \sin(2\pi\omega_i^\top x) \sin(2\pi\omega_i^\top y) \right) \\
&= \sum_{i=1}^N \gamma_i \cos(2\pi\omega_i^\top (x - y)).
\end{aligned}$$

On doit donc avoir $\gamma_i = \beta_i$ et on peut prendre $a_i, b_i \sim \mathcal{N}(0, \beta_i)$ i.i.d., qui satisfont les hypothèses. Alternativement, on peut appliquer une normalisation et imposer que $a_i = \sqrt{\beta_i} \tilde{a}_i$ (resp. b_i) et $\tilde{a}_i, \tilde{b}_i \sim \mathcal{N}(0, 1)$ (car $\text{Var}(a_i) = \text{Var}(\sqrt{\gamma_i} \tilde{a}_i) = \gamma_i \text{Var}(\tilde{a}_i) = \gamma_i$).

Dans le cas spécifique d'une quadrature de type Monte Carlo, on a $\beta_i = 1/N$ et $\omega_i \sim p$.

La distribution p peut être déterminée par la première identité (théorème de Bochner) qui relie $K(x, y)$ et $p(\omega)$ par la transformée de Fourier. On a directement $p(\omega) = \mathcal{F}^{-1}[K](\omega)$.

Si on prend le cas où on choisit un noyau de corrélation Gaussien n -dimensionnel anisotropique avec une matrice de corrélation $\Sigma = \sigma\sigma^\top$,

$$K(x, y) = e^{-2\pi^2(x-y)^\top \Sigma (x-y)},$$

alors

$$p(\omega) = \mathcal{F}^{-1}[K](\omega) = \frac{1}{\det(\sigma)(\sqrt{2\pi})^n} e^{-\frac{1}{2}\omega^\top \Sigma^{-1} \omega},$$

et donc $\omega_i \sim \mathcal{N}(0, \Sigma)$.

Dans ce cas, on peut aussi prendre une quadrature déterministe de Gauss-Hermite pour le choix des poids β_i et des noeuds ω_i , puisqu'on intègre contre un poid Gaussien, ce qui peut garantir une approximation plus précise que Monte Carlo en utilisant un N plus bas.

On va donc définir notre processus aléatoire par

$$D(x, \omega) = D_0(x) + \delta D(x, \omega).$$

On retrouve donc une structure similaire au développement K-L de la forme

$$D(x, \omega) \approx D_0(x) + \sum_{i=1}^{2N} \sqrt{\beta_i} \xi_i(\omega) \phi_i(x)$$

où $\xi_i(\omega) \sim \mathcal{N}(0, 1)$. La suite de la dérivation est la même que dans les sections précédentes, les matrices A_i sont calculées différemment. Les ξ_i sont aussi i.i.d. Gaussiens, donc l'utilisation des polynômes d'Hermite dans le chaos polynomial est toujours valide. L'avantage : à aucun moment n'a-t-on eu besoin d'assembler une matrice dense non-locale, ni à résoudre un problème de valeurs propres.

A blabla